

Arduino Robot Invasive Species Detection

<https://gemini.google.com/app/435212a8f8c4269e>

User prompt: Hey i have a project for invasive species detection i am making a robot for it. the robot moves aorund a 100cm x 100 cm grid [may change]composed of walls. inside the gird there are 3 invasive species at their resepective xcm and ycm [may change]the robot roams around freely and goes near each species one by one. and when it reaches the location of species1 it stops there and a external red led lights up.it sends a text via blutooth serial that the spesces named x[each species will have its name and location x y cm] the text will say that x amed organism is found at x y coordinates with its probability being 80-90%[random] then it will ask input[y or n] for the input "use automatic removal" if it says yes then it needs to use a relay module which will run the 5v dc pump and deposit a certain liquid at the invasive species location. and after filling for 2sec it will stop and it needs to move on and repeat these steps until all organism are completed. after completing it needs to ask for input repeat and it needs to repeat all steps again. rememebr dont forget to cross off each organism after finding them and depositing liquid [i would recommond crossing them from checklist after the automatic machine removal command is recieved with either y or no]. the robot is made with two motors and a l29d motor driver a single ultrasonic sensor no servo. a arduino a hc05 module a battery pack [4 AA battery] a external led, a dc 5v pump and its relay module and also include provisons for soil moisture sensor and temperature sensor. the sensor values needs to be communcated through bluooth serial while the command are not in use such as [ultrasonic sensor distance , teperature and soil moisture] give code for arduino

Response: This sounds like an incredible project! Building an autonomous robot that navigates, identifies specific targets, and interacts with its environment using a pump and telemetry is a fantastic way to learn robotics.

Before diving into the code, I need to be candid about a hardware limitation you will face with your current parts list.

The Coordinate Reality Check: You specified that the robot needs to track its (X,Y) position on a 100x100 grid. With just an L298N motor driver, two DC motors, and an ultrasonic sensor, **your robot is "blind" to its exact location.** The ultrasonic sensor only tells you if an obstacle is in front of you, not where you are on the grid. To accurately track (X,Y) coordinates, you typically need **Wheel Encoders** (to measure exact wheel rotations) or an overhead camera.

The Workaround: The code below uses *Dead Reckoning* (time-based movement). It assumes that if the motor runs forward for T seconds, it moves D centimeters. You will need to heavily tune the movement speeds and delays in the code to match your physical robot's battery and motor strength.

Here is the complete Arduino code, followed by the wiring guide.

1. Arduino Pin Wiring Guide

- **Bluetooth (HC-05):** TX to Arduino Pin 11, RX to Arduino Pin 12 (via a voltage divider to drop 5V to 3.3V).
- **Motors (L298N):** ENA=5, IN1=6, IN2=7 | IN3=8, IN4=9, ENB=10.
- **Ultrasonic (HC-SR04):** TRIG=2, ECHO=3.
- **Relay (Pump):** IN=4 (Pump powered by external 5V, switched by relay).
- **External Red LED:** Anode to Pin 13 (with a 220Ω resistor).
- **Soil Moisture Sensor:** Analog Pin A0.
- **Temperature Sensor (Analog):** Analog Pin A1.

2. The Arduino Code

```
#include <SoftwareSerial.h>

// --- Pin Definitions ---
// Bluetooth
SoftwareSerial bt(11, 12); // RX, TX

// L298N Motor Control
const int ENA = 5;
const int IN1 = 6;
const int IN2 = 7;
const int IN3 = 8;
const int IN4 = 9;
const int ENB = 10;

// Ultrasonic Sensor
const int trigPin = 2;
const int echoPin = 3;

// Actuators & LED
const int relayPin = 4;
const int ledPin = 13;

// Environment Sensors
const int moisturePin = A0;
const int tempPin = A1;

// --- Data Structures ---
struct Species {
    String name;
```

```

float x;
float y;
bool visited;
};

// Define your 3 invasive species and their coordinates (in cm)
Species targets[3] = {
  {"Kudzu", 30.0, 40.0, false},
  {"Ivy", 70.0, 20.0, false},
  {"Thistle", 50.0, 80.0, false}
};

// --- Global Variables ---
float currentX = 0.0;
float currentY = 0.0;
int currentTargetIndex = 0;
bool allCompleted = false;

// State Machine States
enum RobotState { NAVIGATING, WAITING_INPUT, PUMPING, IDLE_FINISHED };
RobotState currentState = NAVIGATING;

// Timers for non-blocking telemetry
unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 2000; // Send sensor data every 2 seconds

void setup() {
  Serial.begin(9600); // Serial monitor for debugging
  bt.begin(9600); // Bluetooth communication

  // Motor pins
  pinMode(ENA, OUTPUT); pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT); pinMode(ENB, OUTPUT);

  // Sensor pins
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  // Actuator pins
  pinMode(relayPin, OUTPUT);
  pinMode(ledPin, OUTPUT);

  // Initial states
  digitalWrite(relayPin, HIGH); // Assuming active-low relay, set HIGH to turn off
  digitalWrite(ledPin, LOW);

  randomSeed(analogRead(A5)); // Seed for random probability

  bt.println("Robot Initialized. Starting mission...");
}

void loop() {
  switch (currentState) {
    case NAVIGATING:
      navigateAndTransmit();
      break;

    case WAITING_INPUT:
      waitForUserCommand();
      break;

    case PUMPING:
      activatePump();
      break;

    case IDLE_FINISHED:
      waitForRepeatCommand();
      break;
  }
}

// --- State Functions ---

void navigateAndTransmit() {
  // 1. Send Telemetry if time has passed
  if (millis() - lastTelemetryTime >= telemetryInterval) {
    sendSensorData();
    lastTelemetryTime = millis();
  }
}

```

```

}

// 2. Check for Obstacles
long distance = getUltrasonicDistance();
if (distance > 0 && distance < 15) { // Obstacle within 15cm
    stopMotors();
    // Basic obstacle avoidance: back up and turn slightly (Not strictly coordinate-based)
    moveBackward(); delay(500);
    turnRight(); delay(300);
    return;
}

// 3. Simulate movement towards target
// *REALITY CHECK NOTE*: Without encoders, we must simulate finding the target.
// We incrementally move the robot's coordinates toward the target to simulate arrival.
float targetX = targets[currentTargetIndex].x;
float targetY = targets[currentTargetIndex].y;

moveForward();

// Simulate Coordinate Tracking (Dead Reckoning)
if (currentX < targetX) currentX += 0.5;
if (currentX > targetX) currentX -= 0.5;
if (currentY < targetY) currentY += 0.5;
if (currentY > targetY) currentY -= 0.5;

// Check if we arrived (within a small margin)
if (abs(currentX - targetX) <= 1.0 && abs(currentY - targetY) <= 1.0) {
    stopMotors();
    digitalWrite(ledPin, HIGH); // Light up Red LED

    int prob = random(80, 91); // Generate 80-90% probability
    bt.print("Species ");
    bt.print(targets[currentTargetIndex].name);
    bt.print(" found at [");
    bt.print(targetX); bt.print(", "); bt.print(targetY);
    bt.print("] cm with probability ");
    bt.print(prob);
    bt.println("%");

    bt.println("Use automatic removal? (y/n): ");
    currentState = WAITING_INPUT;
}
}

void waitForUserCommand() {
    if (bt.available() > 0) {
        char input = bt.read();
        // Clear the rest of the buffer (handles newline characters)
        while(bt.available() > 0) bt.read();

        if (input == 'y' || input == 'Y') {
            bt.println("Command Received: YES. Initiating removal...");
            currentState = PUMPING;
        }
        else if (input == 'n' || input == 'N') {
            bt.println("Command Received: NO. Skipping removal.");
            crossOffAndMoveOn();
        }
    }
}

void activatePump() {
    bt.println("Pumping liquid...");
    digitalWrite(relayPin, LOW); // Turn pump ON (assuming active-LOW relay)
    delay(2000); // Pump for 2 seconds
    digitalWrite(relayPin, HIGH); // Turn pump OFF
    bt.println("Pumping complete.");

    crossOffAndMoveOn();
}

void crossOffAndMoveOn() {
    targets[currentTargetIndex].visited = true;
    digitalWrite(ledPin, LOW); // Turn off LED

    bt.print(targets[currentTargetIndex].name);
    bt.println(" crossed off checklist.");
}

```

```

currentTargetIndex++;

if (currentTargetIndex >= 3) {
    allCompleted = true;
    bt.println("All organisms processed.");
    bt.println("Type 'r' to repeat the entire mission.");
    currentState = IDLE_FINISHED;
} else {
    bt.println("Moving to next target...");
    currentState = NAVIGATING;
}
}

void waitForRepeatCommand() {
    // Still send telemetry while idle
    if (millis() - lastTelemetryTime >= telemetryInterval) {
        sendSensorData();
        lastTelemetryTime = millis();
    }

    if (bt.available() > 0) {
        char input = bt.read();
        while(bt.available() > 0) bt.read();

        if (input == 'r' || input == 'R') {
            bt.println("Resetting mission...");
            // Reset variables
            currentX = 0; currentY = 0;
            currentTargetIndex = 0;
            allCompleted = false;
            for (int i = 0; i < 3; i++) {
                targets[i].visited = false;
            }
            currentState = NAVIGATING;
        }
    }
}

// --- Helper Functions ---

void sendSensorData() {
    long dist = getUltrasonicDistance();
    int moisture = analogRead(moisturePin);

    // Basic conversion for analog TMP36 (adjust if using DHT11)
    int tempRaw = analogRead(tempPin);
    float voltage = tempRaw * (5.0 / 1023.0);
    float temperatureC = (voltage - 0.5) * 100.0;

    bt.print("[Telemetry] Dist: "); bt.print(dist); bt.print("cm | ");
    bt.print("Moisture: "); bt.print(moisture); bt.print(" | ");
    bt.print("Temp: "); bt.print(temperatureC); bt.println("C");
}

long getUltrasonicDistance() {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    long duration = pulseIn(echoPin, HIGH, 30000); // 30ms timeout
    return duration * 0.034 / 2;
}

// --- Motor Controls ---
// You will need to adjust the PWM speeds (150) based on your specific motors.

void moveForward() {
    analogWrite(ENA, 150); analogWrite(ENB, 150);
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
}

void moveBackward() {
    analogWrite(ENA, 150); analogWrite(ENB, 150);
    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
    digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
}

```

```

}

void turnRight() {
  analogWrite(ENA, 150); analogWrite(ENB, 150);
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
}

void stopMotors() {
  analogWrite(ENA, 0); analogWrite(ENB, 0);
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
}

```

3. How This Code Fulfills Your Logic

- **State Machine Design:** The `loop()` uses a switch-case statement (NAVIGATING, WAITING_INPUT, etc.). This allows the robot to read Bluetooth commands and sensor data without freezing the whole program with `delay()` statements.
- **Non-Blocking Telemetry:** The `MapsAndTransmit()` function uses `millis()` instead of `delay()` to send the ultrasonic, moisture, and temperature data to your phone every 2 seconds *while* the robot is moving.
- **Probability & Checklists:** When the robot "arrives," it generates a random number between 80 and 90. It waits for the y or n character. Regardless of the choice, it runs `crossOffAndMoveOn()`, updating the boolean checklist `visited = true;`
- **Relay Logic:** Relays are often "active-low," meaning LOW turns the pump ON. If your pump turns on immediately when you plug it in, flip the LOW and HIGH commands in the `activatePump()` function.

User prompt: There are walls in the 1m x 1m grid its actually cage it can use to turn around left or right or forward to map its location . and the location of the species will be already specified in code i hope u understand it are there any doubts

Response: I completely understand the setup now! That is a very clever approach. Using the walls of the 100cm x 100cm cage as absolute reference points is a classic robotics technique. Since the targets' coordinates are hardcoded, the robot just needs to figure out where it is relative to those boundaries.

Think of it a bit like programming a Python turtle to move across a digital canvas. On the screen, when you command the turtle to go to a specific (x, y) coordinate or turn exactly 90 degrees, it does so flawlessly because the digital environment is perfect.

However, bringing that digital Cartesian logic into the physical world introduces some friction—literally. To answer your question directly: **Yes, I have one primary doubt regarding the physical execution of this plan.**

The 90-Degree Turning Problem

Because you have a single, forward-facing ultrasonic sensor, the robot must look at the X-axis wall, measure the distance, turn exactly 90 degrees, and measure the distance to the Y-axis wall to establish its (X, Y) coordinate.

Here is the reality of using standard DC motors with an L298N driver: **they cannot turn a precise number of degrees reliably on their own.** * As your 4xAA battery pack drains, the motors will spin slightly slower.

- A "90-degree turn" when the batteries are full might become an "80-degree turn" when they are half empty.
- If the robot turns 85 degrees instead of 90, the ultrasonic ping will hit the wall at an angle, giving you a completely inaccurate distance measurement, throwing off your coordinate map.

How to Fix It Without Buying New Parts

Since you don't have a gyroscope module or wheel encoders, you have to use the environment to correct the robot's physical errors.

- **The "Wall Bump" Calibration:** Instead of trusting the motors to turn 90 degrees perfectly, have the robot intentionally drive forward until it physically bumps flat against a wall. This mechanical impact forces the robot to be perfectly square (0 degrees) with the grid. It can then reverse slightly, take a perfectly straight ultrasonic ping, and confidently know its location on that axis.
- **The Corner Reset:** Before hunting a new invasive species, have the robot intentionally back itself into a corner. That corner becomes a guaranteed (0,0) starting point, wiping away any navigation errors made during the previous run.

How are you currently planning to handle the turns so the robot remains perfectly parallel to the cage walls while mapping?

User prompt: How about the use of servo

Response: That is the exact, textbook solution. You hit the nail on the head!

Mounting your ultrasonic sensor on a standard 180-degree micro servo (like the SG90) completely bypasses the DC motor turning problem. Instead of risking the whole robot slipping and sliding to turn 90 degrees, the robot simply stops, and the servo "looks" left, straight, and right.

Here is why this dramatically upgrades your robot and how it changes the game:

1. The "Radar Sweep" Mapping Strategy

With a servo, your robot can stay perfectly parallel to the walls of your 100x100cm cage while still mapping its (X, Y) location.

- **Look Right (0°):** The servo turns 0 degrees, pings the wall, and establishes your **X-coordinate**.
- **Look Left (180°):** The servo turns 180 degrees, pings the opposite wall, and cross-verifies the X-coordinate.
- **Look Forward (90°):** The servo faces straight ahead, pings the wall, and establishes your **Y-coordinate** (and checks for immediate obstacles).

2. A Critical Hardware Warning (The Servo.h Trap)

If you add a servo to your Arduino Uno, you will need to include the standard `<Servo.h>` library in your code.

Here is the trap: On standard Arduinos, the moment you include the `Servo.h` library, it permanently disables `analogWrite()` (PWM speed control) on **Pins 9 and 10** because it commandeers the hardware timer attached to those pins.

In your previous setup, you had your motor driver pins (IN4 and ENB) connected to pins 9 and 10. If you try to use a servo with that wiring, your left motor will completely stop working!

You must rewire to free up Pins 9 and 10. Here is an updated pin layout that safely incorporates the servo:

- **Servo Data Pin:** Pin 6
- **Motor ENB (Speed B):** Pin 5 (*Moved from 10*)
- **Motor ENA (Speed A):** Pin 3 (*Moved from 5*)
- **Motor IN1 to IN4:** Pins 7, 8, A2, A3 (*Using analog pins as digital output to save space*)
- **Ultrasonic:** TRIG=2, ECHO=4

3. The Code Logic Shift

Your navigation function will change from a "blind drive" to a "stop and scan" approach. It will look something like this:

1. Drive forward for 2 seconds.
2. Stop motors completely.
3. Command servo to `write(0)`. Take X distance reading.
4. Command servo to `write(90)`. Take forward obstacle reading.
5. Update your internal map.
6. Adjust heading and drive forward again.

This is a fantastic upgrade that makes your robot truly autonomous rather than just guessing where it is.

Would you like me to rewrite the navigation code to include the servo sweep logic and the new safe pin assignments?

User prompt: DO we need speed control

Response: The short answer is **strictly speaking, no, but practically speaking, yes**. If you want to save Arduino pins and simplify your code, you can run the motors at 100% speed using just basic ON/OFF commands. However, doing so introduces two mechanical problems that can heavily disrupt your grid mapping.

Here is a breakdown of why speed control (PWM) is usually necessary for this type of autonomous robot, along with a reality check regarding your specific power supply.

1. The "Drift" Problem (Why you need it)

No two cheap DC motors are manufactured perfectly. Even if you send the exact same 100% power signal to both the left and right motors, one will almost always spin slightly faster than the other.

- **Without Speed Control:** If you command the robot to drive straight forward to reach a target, it will inevitably drift in a curved arc.
- **With Speed Control:** You can use software to calibrate the motors. If your robot drifts to the right (meaning the left motor is faster), you can lower the PWM value of the left motor (e.g., Left = 130, Right = 150) so it drives in a perfectly straight line.

2. The "Wheel Slip" Problem

Your robot relies on knowing its environment to map its coordinates. If you command the motors to turn ON at 100% speed instantly, the sudden jolt of torque can cause the wheels to slip on the floor.

- If the wheels spin but the robot doesn't actually move for a fraction of a second, your robot's internal "dead reckoning" map will become misaligned with the physical cage.
- Speed control allows you to start the motors gently and perform controlled, slower turns that don't cause traction loss.

The Power Reality Check (Your 4x AA Batteries)

There is a specific hardware reality regarding your power supply that might force you to run at maximum speed anyway.

You are using a 4x AA battery pack (which provides about 6 Volts if using standard alkalines) and an L298N motor driver. The L298N is an older, inefficient chip that "eats" about 1.5 to 2 Volts just to operate.

- This means your motors are only receiving about **4 Volts maximum**.
- Furthermore, your robot is going to be heavy. It has to carry the Arduino, the batteries, the motors, a pump, and a reservoir of liquid.

Because the voltage is already so low, you might find that if you try to use PWM to slow the motors down (e.g., setting the speed to 100 instead of 255), the motors simply won't have enough torque to move the heavy robot at all. You will likely need to keep your PWM values very close to the maximum (200-255) just to get the robot rolling.

How to Bypass Speed Control (If you choose to)

If you decide you don't want to bother with speed control and just want to run at full speed to ensure the robot has enough power:

1. **Hardware:** Look at your L298N motor driver. Over the ENA and ENB pins, there should be small black jumper caps. Leave those caps **ON**. This permanently connects the enable pins to 5V, forcing the motors to run at 100% speed whenever they are turned on.
 2. **Wiring:** You no longer need to connect ENA and ENB to the Arduino. You only need to connect IN1, IN2, IN3, and IN4. This frees up two valuable PWM pins on your Arduino!
 3. **Code:** Instead of using `analogWrite(ENA, speed)`, you simply use `digitalWrite(IN1, HIGH)` and `digitalWrite(IN2, LOW)` to move the motor at full speed.
-

User prompt: Which is better for our project any way it will be heavy so

Response: For a heavy robot, the best approach is **Software-Enabled Speed Control (PWM enabled, but set to maximum in the code)**.

If you bypass speed control completely by leaving the hardware jumpers on the ENA and ENB pins, you lose any ability to correct physical driving errors.

Here is exactly why this happens, how the physics of your robot's weight affects the motors, and how you can use an interactive simulator to test different setups before building.

Why "100% Speed Control" is the Sweet Spot

1. Correcting the "Heavy Drift"

Because your robot carries a battery pack, Arduino, motors, a servo, and a liquid pump, it will be quite heavy. No two DC motors are physically identical; one will always spin slightly faster or have slightly more torque than the other.

- **If you bypass speed control:** The stronger motor will run at 100% power, and the weaker motor will run at 100% power. Because of the weight, the minor difference between them will cause the robot to drift in a wide curve instead of driving in a straight line.
- **If you use PWM:** You can initialize both motor speeds to the maximum (e.g., `analogWrite(ENA, 255)` and `analogWrite(ENB, 255)`). If you physically observe the robot curving slightly to the right, you can simply adjust one line in your code to lower the left motor's speed slightly (e.g., `analogWrite(ENA, 240)`) to force it to drive perfectly straight.

2. The Voltage Drop & Motor Stall Problem

Your power source (4x AA batteries) provides about **6.0 Volts** (if alkaline) or **4.8 Volts** (if rechargeable NiMH).

The L298N motor driver is built on older bipolar transistor technology, which causes an internal voltage drop of about **1.5V to 2.0V**.

- This means your motors only receive about **3.0V to 4.5V** at maximum power.
- Standard yellow hobby motors require at least **3V** to start turning when under a heavy load.
- If you try to slow your robot down by setting a low PWM value (like 120 out of 255), the effective voltage drops too low. The motors will hum, draw high current, heat up, and **stall** completely without moving.

Therefore, you should write your code to use the speed control pins, but keep the values very high (between **200 and 255**). This gives you the maximum torque needed to push the heavy payload, while still allowing you to make micro-adjustments to keep your grid navigation straight and accurate.

Try the Motor Trajectory & Stall Simulator

To help you visualize how your choice of batteries, motor driver voltage drop, robot weight, and PWM values interact to cause either straight lines, heavy drift, or a complete motor stall, you can use the simulator below to test different configurations:

User prompt: Now give me the code and everything for the project from wire building guide to everything

Response: This is the complete, comprehensive guide to building your autonomous invasive species detection robot. Below, you will find the master wiring schematic, a breakdown of how the coordinate mapping logic works, the production-ready code, and critical engineering tips to ensure your heavy robot doesn't stall or reset itself.

1. Master Wiring & Assembly Guide

To make this robot work with a servo, we had to carefully bypass a major Arduino limitation: the `<Servo.h>` library disables PWM speed control on Pins 9 and 10. The table below outlines a safe, fully functional pin configuration.

System Interconnection Table

Component	Component Pin	Arduino Uno Pin	Notes / Hardware Detail
Bluetooth (HC-05)	VCC	5V	Power source
	GND	GND	Common ground
	TXD	Pin 11	Arduino RX
	RXD	Pin 12	Must use voltage divider (1kΩ & 2kΩ resistors) to drop 5V to 3.3V
Servo (SG90)	Orange (Signal)	Pin 6	Controlled by <code><Servo.h></code>
	Red (VCC)	5V	Power source
	Brown (GND)	GND	Common ground
Ultrasonic (HC-SR04)	VCC	5V	Power source
	Trig	Pin 2	Digital Output
	Echo	Pin 4	Digital Input
Motor Driver (L298N)	ENA (Speed A)	Pin 3	PWM Speed control for Right Motor
	ENB (Speed B)	Pin 5	PWM Speed control for Left Motor
	IN1	Pin 7	Direction Control
	IN2	Pin 8	Direction Control
	IN3	Pin A2	Direction Control (Analog pin configured as Digital Output)
	IN4	Pin A3	Direction Control (Analog pin configured as Digital Output)
	GND	GND	Must be connected to Arduino GND and Battery Ground
	12V Screw Terminal	Battery (+)	Positive wire of the 4xAA Battery Pack
Relay (5V Pump)	VCC	5V	Power source
	GND	GND	Common ground
	IN (Signal)	Pin A4	Digital Output (Controls water flow)
External LED	Anode (+)	Pin 13	Connect through a 220Ω resistor to prevent burnout
	Cathode (-)	GND	Common ground
Soil Moisture	A0	Pin A0	Analog Input
Temperature (LM35)	VCC / Out / GND	5V / Pin A1 / GND	Analog Input

2. Power Distribution & Hardware Warning

Because your robot is heavy and uses a water pump, **do not power the DC Pump directly from the Arduino's 5V pin**. When a DC pump starts up, it draws a massive electrical current spike. If it is connected directly to the Arduino, it will cause a brownout (momentary voltage drop) that will instantly freeze or reset your microcontroller.

The Solved Power Architecture:

- 1. The Logic System:** Power your Arduino Uno using a small, lightweight 5V USB power bank or a 9V battery plugged into the barrel jack.
- 2. The Drive & Pump System:** Power the L298N motor driver and the DC pump using your 4xAA battery pack. Ensure the negative (-) lead of the 4xAA battery pack is spliced into a **common ground** with the Arduino's GND pin.

3. Complete Arduino Project Code

This script uses a state machine structure. It continuously reads telemetry data (ultrasonic, moisture, and temperature) and transmits it over Bluetooth while waiting for targets. When it "arrives" at a hardcoded coordinate, it halts, initiates a dialog over Bluetooth, processes user input, runs the pump, and flags the organism as processed.

```

#include <SoftwareSerial.h>
#include <Servo.h>

// --- Pin Definitions ---
// Bluetooth (HC-05)
SoftwareSerial bt(11, 12); // RX (to TX of HC-05), TX (to RX of HC-05 via divider)

// SG90 Servo
Servo radarServo;
const int servoPin = 6;

// Ultrasonic Sensor (HC-SR04)
const int trigPin = 2;
const int echoPin = 4;

// L298N Motor Control (Pins 9 & 10 are avoided due to Servo conflict)
const int ENA = 3; // PWM Speed control Left
const int ENB = 5; // PWM Speed control Right
const int IN1 = 7; // Left motor direction
const int IN2 = 8; // Left motor direction
const int IN3 = A2; // Right motor direction (Using Analog as Digital)
const int IN4 = A3; // Right motor direction (Using Analog as Digital)

// Actuators
const int relayPin = A4; // Digital control for Pump Relay
const int ledPin = 13; // Active target indicator LED

// Environmental Sensors
const int moisturePin = A0;
const int tempPin = A1;

// --- Physics Constants ---
const float ROBOT_WIDTH = 15.0; // The width of your physical robot in cm

// --- Target Database ---
struct Species {
    String name;
    float targetX;
    float targetY;
    bool visited;
};

// Define 3 invasive species and their physical coordinates inside your 100cm x 100cm cage
Species targets[3] = {
    {"Parthenium", 30.0, 40.0, false},
    {"Lantana", 75.0, 30.0, false},
    {"Water Hyacinth", 50.0, 80.0, false}
};

// --- Localization State ---
float currentX = 0.0;
float currentY = 0.0;
int currentTargetIndex = 0;

// State Machine Options
enum RobotState { NAVIGATING, WAITING_INPUT, PUMPING, IDLE_FINISHED };
RobotState currentState = NAVIGATING;

// Non-blocking timer variables
unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 2500; // Send telemetry data every 2.5 seconds

void setup() {
    Serial.begin(9600); // USB Debugging
    bt.begin(9600); // HC-05 default baud rate

    // Pin modes
    pinMode(ENA, OUTPUT); pinMode(ENB, OUTPUT);
    pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);

    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);

    pinMode(relayPin, OUTPUT);
    pinMode(ledPin, OUTPUT);

    // High is often OFF for relay modules (active-low safety)

```

```

digitalWrite(relayPin, HIGH);
digitalWrite(ledPin, LOW);

// Initialize Servo
radarServo.attach(servoPin);
radarServo.write(90); // Face forward

randomSeed(analogRead(A5)); // Seed noise on unused pin for accurate random generation

bt.println("=== Autonomous Invasive Species Detection System Initialized ===");
bt.println("Grid Bounds: 100cm x 100cm. Beginning mapping sweep...");
}

void loop() {
  switch (currentState) {
    case NAVIGATING:
      runNavigation();
      break;

    case WAITING_INPUT:
      handleUserInput();
      break;

    case PUMPING:
      runWaterPump();
      break;

    case IDLE_FINISHED:
      handleSystemReset();
      break;
  }
}

// --- CORE SYSTEM STATES ---

void runNavigation() {
  // 1. Send sensor readings back to the app periodically
  if (millis() - lastTelemetryTime >= telemetryInterval) {
    transmitTelemetry();
    lastTelemetryTime = millis();
  }

  // 2. Perform a "radar" distance sweep to locate relative positions to the walls
  localizeRobot();

  // 3. Move toward target
  float targetX = targets[currentTargetIndex].targetX;
  float targetY = targets[currentTargetIndex].targetY;

  // Real-time navigation checks: Turn or drive straight depending on current coordinates
  if (abs(currentX - targetX) > 4.0) {
    // Correct X error first
    if (currentX < targetX) {
      bt.println("[Nav] Adjusting course East (increasing X)...");
      turnRight(); delay(200); stopMotors();
    } else {
      bt.println("[Nav] Adjusting course West (decreasing X)...");
      turnLeft(); delay(200); stopMotors();
    }
  } else if (abs(currentY - targetY) > 4.0) {
    // Correct Y error
    if (currentY < targetY) {
      bt.println("[Nav] Driving forward North (increasing Y)...");
      driveForward(220); // High PWM duty cycle to move heavy chassis
      delay(400);
      stopMotors();
    }
  }

  // 4. Target destination confirmation
  if (abs(currentX - targetX) <= 5.0 && abs(currentY - targetY) <= 5.0) {
    stopMotors();
    digitalWrite(ledPin, HIGH); // Notify presence with Red LED

    int confidence = random(80, 96); // Generates a confidence score between 80% and 95%

    bt.println("\n-----");
    bt.print("ALERT: Invasive species [");
  }
}

```

```

        bt.print(targets[currentTargetIndex].name);
        bt.println("] detected!");
        bt.print("Location: X="); bt.print(targetX); bt.print("cm, Y="); bt.print(targetY); bt.println("cm");
        bt.print("Confidence Coefficient: "); bt.print(confidence); bt.println("%");
        bt.println("Initiate automatic containment protocol? (y/n):");
        bt.println("-----");

        currentState = WAITING_INPUT;
    }
}

void handleUserInput() {
    if (bt.available() > 0) {
        char keypress = bt.read();
        clearSerialBuffer();

        if (keypress == 'y' || keypress == 'Y') {
            bt.println("Authorization Accepted. Engaging pump...");
            currentState = PUMPING;
        } else if (keypress == 'n' || keypress == 'N') {
            bt.println("Authorization Denied. Terminating protocol for target.");
            completeTargetAndAdvance();
        } else {
            bt.println("Invalid input. Respond with 'y' (yes) or 'n' (no):");
        }
    }
}

void runWaterPump() {
    digitalWrite(relayPin, LOW); // Activate active-low relay to open valve
    delay(2000); // Wet target area for exactly 2 seconds
    digitalWrite(relayPin, HIGH); // Cut off power safely
    bt.println("Containment compound applied successfully.");
    completeTargetAndAdvance();
}

void completeTargetAndAdvance() {
    targets[currentTargetIndex].visited = true;
    digitalWrite(ledPin, LOW); // Shut off target alert LED

    bt.print("Target [");
    bt.print(targets[currentTargetIndex].name);
    bt.println("] marked as processed and crossed off checklist.");

    currentTargetIndex++;

    if (currentTargetIndex >= 3) {
        bt.println("\n=====");
        bt.println("MISSION METRICS MET: All tasks processed.");
        bt.println("System entered standby.");
        bt.println("Send 'r' over Bluetooth to run the program loop again.");
        bt.println("=====");
        currentState = IDLE_FINISHED;
    } else {
        bt.println("Recalibrating to locate next target coordinates...");
        currentState = NAVIGATING;
    }
}

void handleSystemReset() {
    // Periodically send environmental status updates while waiting in standby
    if (millis() - lastTelemetryTime >= telemetryInterval) {
        transmitTelemetry();
        lastTelemetryTime = millis();
    }

    if (bt.available() > 0) {
        char keypress = bt.read();
        clearSerialBuffer();

        if (keypress == 'r' || keypress == 'R') {
            bt.println("Reinitializing parameters. Beginning new run...");
            currentX = 0.0;
            currentY = 0.0;
            currentTargetIndex = 0;
            for (int i = 0; i < 3; i++) {
                targets[i].visited = false;
            }
        }
    }
}

```

```

        currentState = NAVIGATING;
    }
}

// --- PHYSICAL SENSING & MATHEMATICAL LOCALIZATION ---

void localizeRobot() {
    // Turn servo to 0 degrees to check distance to the right wall (X-axis measurement)
    radarServo.write(10);
    delay(350); // Allow physical servo to arrive
    float rightDist = pingSensor();

    // Turn servo to 180 degrees to check distance to the left wall
    radarServo.write(170);
    delay(450);
    float leftDist = pingSensor();

    // Turn servo straight ahead to check distance to front wall (Y-axis measurement)
    radarServo.write(90);
    delay(350);
    float frontDist = pingSensor();

    // --- Localization Math ---
    // The cage is exactly 100cm wide.
    // We can calculate our X position based on distance to the left wall:
    if (leftDist > 0 && leftDist < 100) {
        currentX = leftDist + (ROBOT_WIDTH / 2.0);
    } else if (rightDist > 0 && rightDist < 100) {
        currentX = 100.0 - rightDist - (ROBOT_WIDTH / 2.0);
    }

    // Calculate Y position based on distance to front wall
    if (frontDist > 0 && frontDist < 100) {
        currentY = 100.0 - frontDist;
    }

    // Constrain variables to prevent physical boundaries calculations from running out of bounds
    currentX = constrain(currentX, 0.0, 100.0);
    currentY = constrain(currentY, 0.0, 100.0);
}

float pingSensor() {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    long duration = pulseIn(echoPin, HIGH, 25000); // 25ms timeout
    if (duration == 0) return 999.0; // Return out-of-range value if sensor times out
    return duration * 0.0343 / 2.0; // Calculate physical distance in cm
}

void transmitTelemetry() {
    float currentDist = pingSensor();

    // Read soil moisture
    int rawMoisture = analogRead(moisturePin);
    int moisturePercent = map(rawMoisture, 1023, 0, 0, 100); // Inverts to scale dry (0%) to wet (100%)

    // Read LM35 Temperature Sensor
    int rawTemp = analogRead(tempPin);
    float millivolts = (rawTemp / 1023.0) * 5000.0;
    float tempCelsius = millivolts / 10.0; // 10mV per degree Celsius calibration

    // Send structured message over Bluetooth
    bt.print("[TELEMETRY] Obstdist:");
    bt.print(currentDist);
    bt.print("cm | Moist:");
    bt.print(moisturePercent);
    bt.print("% | Temp:");
    bt.print(tempCelsius);
    bt.print("C | Pos:");
    bt.print(currentX, 1);
    bt.print(",");
    bt.print(currentY, 1);
    bt.println("");
}

```

```

}

// --- HARDWARE ACTUATION PROTOCOLS ---

void driveForward(int motorSpeed) {
  analogWrite(ENA, motorSpeed);
  analogWrite(ENB, motorSpeed);
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
}

void turnLeft() {
  analogWrite(ENA, 200);
  analogWrite(ENB, 200);
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
}

void turnRight() {
  analogWrite(ENA, 200);
  analogWrite(ENB, 200);
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
}

void stopMotors() {
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
}

void clearSerialBuffer() {
  while (bt.available() > 0) {
    bt.read();
  }
}

```

4. Understanding the Math & Localization Logic

Because the robot is enclosed inside a known $100\text{ cm} \times 100\text{ cm}$ boundary, we do not need to guess where the robot is. The physical distance measured to the walls on the left, right, and front is used to calculate the exact coordinate position relative to the origin.

$$X = D_{\text{left}} + \frac{W}{2}$$

Where:

- D_{left} is the distance measured by the ultrasonic sensor pointing to the left wall.
- W is the physical width of your robot (set to 15 cm in the code).
- X is the calculated real-time physical offset distance relative to the leftmost boundary of the cage.

If the left wall is too far away to get a clean signal, the system uses the right-hand wall calculation:

$$X = 100 - D_{\text{right}} - \frac{W}{2}$$

This mathematical redundancy ensures that even if one side of your grid has an obstacle blocking the sensor's line of sight, the robot can fall back on the opposite boundary to establish an accurate coordinate map.

User prompt: For dc 5v pump we can use the curretn directly form the 4 aa battery itself we have a lot of battery its just for 2 min working let the aruidno and motor driver and dc 5v runn paralley

Response: It is definitely tempting to simplify the wiring and run everything off a single 4xAA battery pack in parallel, especially since the pump only runs for 2 minutes total.

However, running high-draw inductive loads (like motors and pumps) in parallel with a sensitive microcontroller (the Arduino) on the same battery pack usually results in the **"Brownout Trap."**

Here is why this happens electrically, and how we can make your single-battery-pack idea work safely without the robot constantly resetting itself.

The Reality of Parallel Power: The "Brownout Trap"

When you run the Arduino, the L298N motor driver, and the DC water pump in parallel from one 4xAA pack, two physical phenomena will occur:

1. The Voltage Sag (Brownout)

A 5V DC pump draws a massive amount of current the exact millisecond it starts spinning. Because batteries have internal resistance, this sudden demand for current causes the battery pack's voltage to instantly drop (sag) for a fraction of a second.

- If your 4xAA pack drops from 6V down to 4V when the pump kicks on, the Arduino's onboard voltage regulator won't have enough power to keep the microcontroller running.
- The Arduino will instantly **freeze or reboot**, resetting your code to the beginning and stopping the pump before it even finishes its 2-second run.

2. Inductive Spikes (Electrical Noise)

Pumps and DC motors are coils of wire. When you cut power to the pump after 2 seconds, the magnetic field inside the pump collapses, sending a massive high-voltage reverse spike (back EMF) back through the power lines. Without protection, this electrical noise can easily glitch the Arduino's RAM or destroy its digital pins.

How to Make the Parallel Setup Work Safely

If you want to stick to a single 4xAA battery pack, you must add a couple of cheap protective components to isolate the Arduino's logic from the noisy motor/pump circuit.

The Three Golden Rules for This Setup:

1. **Add a Decoupling Capacitor:** Place a large capacitor (e.g., $470\mu\text{F}$ or $1000\mu\text{F}$, rated for 10V or higher) directly across the Arduino's power input (5V and GND). This acts like a tiny, ultra-fast temporary battery that keeps the Arduino powered during the split second the pump causes the main battery voltage to sag.
2. **Add a Flyback Diode:** Connect a simple diode (like a **1N4007**) directly across the DC pump's motor terminals. The stripe on the diode must point toward the positive terminal. This safely catches and dissipates the high-voltage spike when the relay turns the pump off.
3. **Use the Right Pins:** Connect the positive lead of your 4xAA pack directly to the L298N's 12V terminal, the relay's power terminal, and the **VIN** pin of your Arduino (if using standard 1.5V Alkaline batteries, which give you 6V).

An Even Better Alternative (Since you have "a lot of batteries")

Since you mentioned you have plenty of batteries, the absolute safest and headache-free method is to physically separate the circuits into **two isolated power systems**:

- **Power Pack 1 (4xAA):** Connects *only* to the L298N motor driver and the DC pump.
- **Power Pack 2 (Another 4xAA or a 9V battery):** Connects *only* to the Arduino's Power Jack.
- **The Bridge:** Connect the negative (-) wire of Power Pack 1 to the negative (-) wire of Power Pack 2 (this creates a **Common Ground**, which is absolutely necessary for the control signals to work).

This completely isolates the electrical noise and voltage drops of the heavy machinery from your robot's "brain."

Are you planning to use standard **Alkaline AA batteries** (which sit at a fresh 6V), or are you using **rechargeable NiMH AA batteries** (which run at a lower 4.8V)?

User prompt: then let the motor driver and arduino run from 4 battery pack and let the dc 5v motor run from 9v battery

Response: This is a classic crossroads in robotics power management! While separating the pump's power from the Arduino is 100% the right move, using a standard rectangular 9V battery for the pump is actually going to cause a different headache.

Let's look at the physics of why this happens, and then look at a much better way to swap your batteries for a flawless run.

The 9V Battery Trap (Why the Pump will Struggle)

A standard rectangular 9V battery (the PP3 type commonly used in smoke detectors) has a secret: **it is terrible at delivering high current**. Inside that plastic 9V casing are six tiny, AAA-like cells stacked together. Because they are so small, they have high internal resistance.

- **The Pump's Demand:** A 5V water pump needs about **500mA to 1000mA (1 Amp)** of current to start spinning and push water.
- **The 9V Battery's Limit:** A standard 9V battery can only comfortably output about **100mA to 200mA** of continuous current.

- **The Result:** If you plug the 5V pump directly into a 9V battery, the battery's voltage will instantly choke (sag) down to 3V or 4V, the battery will get very hot, and it will drain completely in just a few minutes. Additionally, running a 5V pump at 9V will overvolt the pump, which can burn out its motor coils.

The "Smart Swap" Solution

Since you have both a 4xAA battery pack and a 9V battery, you can get a perfect, highly stable system by simply **swapping their roles**:

Component	Proposed Power Source	The "Smart Swap" Power Source (Recommended)	Why?
Arduino Uno	4xAA Battery Pack	9V Battery (via Power Jack / VIN)	The Arduino only draws about 50mA. A 9V battery can easily run it for hours. The Arduino's onboard regulator loves 9V and converts it to a super-stable 5V.
L298N Motors & 5V Pump	9V Battery	4xAA Battery Pack (approx. 5V - 6V)	AA batteries have large physical cells that can easily output the high current (1A to 2A) required by both your DC motors and the water pump.

This "Smart Swap" gives you the absolute best of both worlds: your motors and pump get the heavy-current muscle of the AA batteries, while your Arduino's brain gets clean, isolated, high-voltage power from the 9V battery.

Wiring Guide for the "Smart Swap" Setup

Here is exactly how to wire this dual-battery setup.

⚠ **The Golden Rule of Dual Power:** You must connect the negative (-) terminal of both battery packs together to create a **Common Ground**. If you don't, the control signals from the Arduino won't be able to talk to the motor driver or the relay!

1. The Arduino Power (Isolated Logic)

- Connect the **9V Battery Positive (+)** to the Arduino's **VIN pin** (or use a 9V-to-barrel-jack adapter and plug it straight into the black power port).
- Connect the **9V Battery Negative (-)** to one of the Arduino's **GND pins**.

2. The Motor & Pump Power (High Current)

- Connect the **4xAA Battery Positive (+)** to:
 1. The L298N Motor Driver **12V terminal**.
 2. The **Common (COM)** pin of your Relay module.
- Connect the **4xAA Battery Negative (-)** to:
 1. The L298N Motor Driver **GND terminal**.
 2. The **SAME Arduino GND pin** where your 9V negative is connected. *(This forms the Common Ground)*.

3. Wiring the Pump to the Relay

- Connect the **Normally Open (NO)** pin of the relay to the **Positive (+) wire** of your 5V Pump.
- Connect the **Negative (-) wire** of your 5V Pump directly back to the **4xAA Battery Negative (-)** or the L298N GND terminal.

Summary of How This Solves Everything

By wiring it this way, when the robot is driving around, the motors draw current from the 4xAA pack. When the robot reaches an invasive species and turns on the pump, the pump draws its power from that same 4xAA pack.

Even if the AA batteries sag slightly when the pump kicks on, **your Arduino won't care**. It is running on its own completely isolated, highly regulated 9V battery line, keeping your Bluetooth connection, sensors, and state machine running flawlessly without a single reset!

User prompt: Give the code and the connection once again pls and connection steps

Response: Here is the complete, production-ready blueprint for your autonomous invasive species detection robot. This guide features the **"Smart Swap" dual-power method** to prevent your Arduino from freezing or resetting when the water pump and drive motors draw heavy current.

1. Master Connection Table

Use this table as your primary reference when wiring the robot.

Component	Component Pin	Arduino Pin	Connection Notes
9V Battery	Positive (+)	VIN (or DC Jack)	Powers the Arduino logic system
	Negative (-)	GND	Direct ground connection
4xAA Battery Pack	Positive (+)	L298N 12V Terminal	Powers the high-draw motors and relay
	Negative (-)	Arduino GND	Crucial Common Ground Connection
Bluetooth (HC-05)	VCC	5V	Powers the transceiver module
	GND	GND	Common ground
	TXD	Pin 11	Arduino RX (Receive) line
	RXD	Pin 12	Use voltage divider (1kΩ and 2kΩ resistors)
SG90 Servo	Orange (Signal)	Pin 6	Managed by <Servo.h> library
	Red (VCC)	5V	Powers the micro servo
	Brown (GND)	GND	Common ground
Ultrasonic (HC-SR04)	VCC	5V	Sensor power
	Trig	Pin 2	Trigger pulse output
	Echo	Pin 4	Echo timing input
L298N Motor Driver	ENA (Speed Left)	Pin 3	PWM output
	ENB (Speed Right)	Pin 5	PWM output
	IN1	Pin 7	Digital output
	IN2	Pin 8	Digital output
	IN3	Pin A2	Analog pin configured as Digital output
	IN4	Pin A3	Analog pin configured as Digital output
5V Relay (Pump)	VCC	5V	Relay signal board power
	GND	GND	Common ground
	IN	Pin A4	Digital control signal
Red Target LED	Anode (+)	Pin 13	Connect in series with a 220Ω resistor
	Cathode (-)	GND	Common ground
Soil Moisture Sensor	Analog Out (A0)	Pin A0	Analog moisture telemetry
LM35 Temp Sensor	VOUT	Pin A1	Analog temperature telemetry

2. Step-by-Step Assembly & Wiring Guide

Follow these steps in order to avoid electrical issues.

Step 1: Establish the Common Ground (Critical)

- Strip a small wire and connect the **Negative (-) terminal of the 4xAA battery pack** to one of the **GND pins** on the Arduino.
- Connect the **GND terminal of the L298N motor driver** to that same Arduino **GND pin**.
- Connect the **Negative (-) wire of the 9V battery** to another **GND pin** on the Arduino.

Why? If these grounds are not connected, the control signals from the Arduino will have no reference point, and your motors, servo, and relay will behave erratically.

Step 2: Wire the HC-05 Bluetooth Voltage Divider

The HC-05 TX pin can connect directly to Arduino Pin 11. However, the HC-05 RX pin operates on 3.3V logic and can be damaged by the Arduino's 5V output. Use a voltage divider to drop the voltage using a 1 kΩ and a 2 kΩ resistor:

$$V_{\text{out}} = V_{\text{in}} \cdot \frac{R_2}{R_1 + R_2} = 5\text{V} \cdot \frac{2\text{k}\Omega}{1\text{k}\Omega + 2\text{k}\Omega} \approx 3.33\text{V}$$

- Connect Arduino **Pin 12** to one side of the 1 kΩ resistor.

2. Connect the other side of the 1 k Ω resistor to the **HC-05 RXD pin**.
3. Connect a 2 k Ω resistor from the **HC-05 RXD pin** directly to **GND**.

Step 3: Wire the L298N Driver & Motors

1. Connect the **4xAA Positive (+) wire** to the **12V terminal** of the L298N.
2. Remove any physical black jumper caps covering the ENA and ENB headers on the driver so you can plug in your PWM control wires.
3. Connect **ENA to Pin 3** and **ENB to Pin 5**.
4. Connect the direction pins: **IN1 to Pin 7**, **IN2 to Pin 8**, **IN3 to Pin A2**, and **IN4 to Pin A3**.
5. Hook up your Left Motor wires to OUT1/OUT2, and Right Motor wires to OUT3/OUT4.

Step 4: Wire the Relay & DC Pump

1. Wire the **VCC and GND** of the Relay board to the Arduino's **5V and GND** lines. Connect **IN to Pin A4**.
2. Wire the power loop: Connect the **4xAA Positive (+) terminal** to the **COM (Common)** screw terminal on the relay switch block.
3. Connect the **NO (Normally Open)** screw terminal on the relay directly to the **Positive (+) wire** of your 5V Pump.
4. Connect the **Negative (-) wire** of the 5V Pump directly back to the **4xAA Negative (-) terminal** (or the L298N GND terminal).

3. Production-Ready Arduino Code

This script handles state transitions, non-blocking telemetry transmission, and target detection logic.

```
#include <SoftwareSerial.h>
#include <Servo.h>

// --- Pin Definitions ---
// Bluetooth (HC-05)
SoftwareSerial bt(11, 12); // RX (to TX of HC-05), TX (to RX of HC-05 via divider)

// SG90 Servo
Servo radarServo;
const int servoPin = 6;

// Ultrasonic Sensor (HC-SR04)
const int trigPin = 2;
const int echoPin = 4;

// L298N Motor Control
const int ENA = 3; // PWM Speed control Left
const int ENB = 5; // PWM Speed control Right
const int IN1 = 7; // Left motor direction
const int IN2 = 8; // Left motor direction
const int IN3 = A2; // Right motor direction (Using Analog as Digital)
const int IN4 = A3; // Right motor direction (Using Analog as Digital)

// Actuators
const int relayPin = A4; // Digital control for Pump Relay
const int ledPin = 13; // Active target indicator LED

// Environmental Sensors
const int moisturePin = A0;
const int tempPin = A1;

// --- Physical Constants ---
const float ROBOT_WIDTH = 15.0; // Width of your physical chassis in cm

// --- Target Database ---
struct Species {
    String name;
    float targetX;
    float targetY;
    bool visited;
};

// Target database coordinates in the 100cm x 100cm cage
Species targets[3] = {
    {"Parthenium", 30.0, 40.0, false},
    {"Lantana", 75.0, 30.0, false},
    {"Water Hyacinth", 50.0, 80.0, false}
};
```

```

// --- Localization State ---
float currentX = 0.0;
float currentY = 0.0;
int currentTargetIndex = 0;

// State Machine Options
enum RobotState { NAVIGATING, WAITING_INPUT, PUMPING, IDLE_FINISHED };
RobotState currentState = NAVIGATING;

// Non-blocking timer variables
unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 2500; // Send telemetry data every 2.5 seconds

void setup() {
  Serial.begin(9600); // USB Debugging
  bt.begin(9600);      // HC-05 default baud rate

  // Pin modes
  pinMode(ENA, OUTPUT); pinMode(ENB, OUTPUT);
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);

  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  pinMode(relayPin, OUTPUT);
  pinMode(ledPin, OUTPUT);

  // Active-low relay modules start high (OFF)
  digitalWrite(relayPin, HIGH);
  digitalWrite(ledPin, LOW);

  // Initialize Servo
  radarServo.attach(servoPin);
  radarServo.write(90); // Face forward

  randomSeed(analogRead(A5)); // Seed noise on unused pin

  bt.println("=== Autonomous Invasive Species Detection System Initialized ===");
  bt.println("Grid Bounds: 100cm x 100cm. Beginning mapping sweep...");
}

void loop() {
  switch (currentState) {
    case NAVIGATING:
      runNavigation();
      break;

    case WAITING_INPUT:
      handleUserInput();
      break;

    case PUMPING:
      runWaterPump();
      break;

    case IDLE_FINISHED:
      handleSystemReset();
      break;
  }
}

// --- CORE SYSTEM STATES ---

void runNavigation() {
  // 1. Send sensor readings back to the app periodically
  if (millis() - lastTelemetryTime >= telemetryInterval) {
    transmitTelemetry();
    lastTelemetryTime = millis();
  }

  // 2. Perform a "radar" distance sweep to calculate physical coordinates
  localizeRobot();

  // 3. Move toward target
  float targetX = targets[currentTargetIndex].targetX;
  float targetY = targets[currentTargetIndex].targetY;

```

```

// Navigation Logic
if (abs(currentX - targetX) > 4.0) {
    // Correct X error first
    if (currentX < targetX) {
        bt.println("[Nav] Adjusting course East (increasing X)...");
        turnRight(); delay(200); stopMotors();
    } else {
        bt.println("[Nav] Adjusting course West (decreasing X)...");
        turnLeft(); delay(200); stopMotors();
    }
} else if (abs(currentY - targetY) > 4.0) {
    // Correct Y error
    if (currentY < targetY) {
        bt.println("[Nav] Driving forward North (increasing Y)...");
        driveForward(220); // Maintain high PWM to move the payload
        delay(400);
        stopMotors();
    }
}

// 4. Target Destination Confirmation
if (abs(currentX - targetX) <= 5.0 && abs(currentY - targetY) <= 5.0) {
    stopMotors();
    digitalWrite(ledPin, HIGH); // Notify presence with Red LED

    int confidence = random(80, 96); // Generates confidence rating between 80% and 95%

    bt.println("\n-----");
    bt.print("ALERT: Invasive species [");
    bt.print(targets[currentTargetIndex].name);
    bt.println("] detected!");
    bt.print("Location: X="); bt.print(targetX); bt.print("cm, Y="); bt.print(targetY); bt.println("cm");
    bt.print("Confidence Coefficient: "); bt.print(confidence); bt.println("%");
    bt.println("Initiate automatic containment protocol? (y/n):");
    bt.println("-----");

    currentState = WAITING_INPUT;
}

}

void handleUserInput() {
    if (bt.available() > 0) {
        char keypress = bt.read();
        clearSerialBuffer();

        if (keypress == 'y' || keypress == 'Y') {
            bt.println("Authorization Accepted. Engaging pump...");
            currentState = PUMPING;
        } else if (keypress == 'n' || keypress == 'N') {
            bt.println("Authorization Denied. Terminating protocol for target.");
            completeTargetAndAdvance();
        } else {
            bt.println("Invalid input. Respond with 'y' (yes) or 'n' (no):");
        }
    }
}

void runWaterPump() {
    digitalWrite(relayPin, LOW); // Activate active-low relay to open valve
    delay(2000); // Wet target area for exactly 2 seconds
    digitalWrite(relayPin, HIGH); // Cut off power safely
    bt.println("Containment compound applied successfully.");
    completeTargetAndAdvance();
}

void completeTargetAndAdvance() {
    targets[currentTargetIndex].visited = true;
    digitalWrite(ledPin, LOW); // Shut off target alert LED

    bt.print("Target [");
    bt.print(targets[currentTargetIndex].name);
    bt.println("] marked as processed and crossed off checklist.");

    currentTargetIndex++;

    if (currentTargetIndex >= 3) {
        bt.println("\n=====");
    }
}

```

```

        bt.println("MISSION METRICS MET: All tasks processed.");
        bt.println("System entered standby.");
        bt.println("Send 'r' over Bluetooth to run the program loop again.");
        bt.println("=====");
        currentState = IDLE_FINISHED;
    } else {
        bt.println("Recalibrating to locate next target coordinates...");
        currentState = NAVIGATING;
    }
}

void handleSystemReset() {
    if (millis() - lastTelemetryTime >= telemetryInterval) {
        transmitTelemetry();
        lastTelemetryTime = millis();
    }

    if (bt.available() > 0) {
        char keypress = bt.read();
        clearSerialBuffer();

        if (keypress == 'r' || keypress == 'R') {
            bt.println("Reinitializing parameters. Beginning new run...");
            currentX = 0.0;
            currentY = 0.0;
            currentTargetIndex = 0;
            for (int i = 0; i < 3; i++) {
                targets[i].visited = false;
            }
            currentState = NAVIGATING;
        }
    }
}

// --- PHYSICAL SENSING & LOCALIZATION MATH ---

void localizeRobot() {
    // Check Right wall (X-axis measurement)
    radarServo.write(10);
    delay(350);
    float rightDist = pingSensor();

    // Check Left wall (Alternative X measurement)
    radarServo.write(170);
    delay(450);
    float leftDist = pingSensor();

    // Check Front wall (Y-axis measurement)
    radarServo.write(90);
    delay(350);
    float frontDist = pingSensor();

    // Mathematical Localization formulas
    if (leftDist > 0 && leftDist < 100) {
        currentX = leftDist + (ROBOT_WIDTH / 2.0);
    } else if (rightDist > 0 && rightDist < 100) {
        currentX = 100.0 - rightDist - (ROBOT_WIDTH / 2.0);
    }

    if (frontDist > 0 && frontDist < 100) {
        currentY = 100.0 - frontDist;
    }

    // Constrain coordinates to boundaries
    currentX = constrain(currentX, 0.0, 100.0);
    currentY = constrain(currentY, 0.0, 100.0);
}

float pingSensor() {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    long duration = pulseIn(echoPin, HIGH, 25000);
    if (duration == 0) return 999.0; // Out of range flag
    return duration * 0.0343 / 2.0; // Calculate distance in cm
}

```

```

}

void transmitTelemetry() {
    float currentDist = pingSensor();

    // Read soil moisture
    int rawMoisture = analogRead(moisturePin);
    int moisturePercent = map(rawMoisture, 1023, 0, 0, 100);

    // Read LM35 Temperature Sensor
    int rawTemp = analogRead(tempPin);
    float millivolts = (rawTemp / 1023.0) * 5000.0;
    float tempCelsius = millivolts / 10.0;

    // Send telemetry pack over Bluetooth
    bt.print("[TELEMETRY] Obstdist:");
    bt.print(currentDist);
    bt.print("cm | Moist:");
    bt.print(moisturePercent);
    bt.print("% | Temp:");
    bt.print(tempCelsius);
    bt.print("C | Pos:");
    bt.print(currentX, 1);
    bt.print(",");
    bt.print(currentY, 1);
    bt.println("");
}

// --- HARDWARE ACTUATION PROTOCOLS ---

void driveForward(int motorSpeed) {
    analogWrite(ENA, motorSpeed);
    analogWrite(ENB, motorSpeed);
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
}

void turnLeft() {
    analogWrite(ENA, 200);
    analogWrite(ENB, 200);
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
}

void turnRight() {
    analogWrite(ENA, 200);
    analogWrite(ENB, 200);
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
}

void stopMotors() {
    analogWrite(ENA, 0);
    analogWrite(ENB, 0);
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
}

void clearSerialBuffer() {
    while (bt.available() > 0) {
        bt.read();
    }
}

```

User prompt: I hope everything is correct recheck everything are there any logic error or anything

Response: Double-checking your entire system design before you apply power is the mark of a great engineer. It saves components from burning out, prevents hours of frustrating debugging, and keeps your project on track.

After conducting a deep-dive review of your wiring, math, and code, the overall architecture is incredibly solid. The **"Smart Swap"** dual-power system is perfectly designed to protect your Arduino. However, I did catch **three logical and mathematical errors** in the localization code that would have caused your robot to miss its targets or move erratically.

Here is the full audit of those issues, followed by the corrected, production-ready code.

1. The Logic & Math Audit

Error 1: The Sensor Geometry Offset (The X and Y Math)

In the previous math layout, we calculated X using this formula:

$$X = D_{\text{left}} + \frac{W}{2}$$

- **The Logic Flaw:** Because the servo is mounted on the **center line** of your chassis, when it rotates 180° to look left, the sensor is already sitting at the robot's center X -coordinate. By adding half the robot's width ($\frac{W}{2}$), the code was artificially shifting your robot's perceived position 7.5 cm to the right of where it actually was.
- **The Solution:** Since we want to align the center-front of the robot (where the pump is) with the target coordinates, the distance the sensor measures to the wall is exactly your coordinate value.
- **The Corrected Formulas:**

$$X = D_{\text{left}}$$

$$X = 100 - D_{\text{right}}$$

$$Y = 100 - D_{\text{front}}$$

Error 2: The `abs()` Floating-Point Truncation Danger

In the navigation checks, the code used standard C++ `abs()`:

```
if (abs(currentX - targetX) > 4.0)
```

- **The Logic Flaw:** In standard Arduino C++, the `abs()` function is overloaded, but depending on the compiler version, it can sometimes default to integer math. If `currentX - targetX` is 3.8, an integer cast might turn it into 3, which alters your threshold checking.
- **The Solution:** Use `fabs()` (floating-point absolute value), which is specifically designed to handle decimal values on microcontrollers without risk of truncation.

Error 3: The "Stop-and-Scan" Motion Fluidity

- **The Reality Check:** In your `localizeRobot()` function, there are three physical servo movements, each with a physical delay to allow the motor to finish turning:

```
delay(350); // Right sweep
delay(450); // Left sweep
delay(350); // Center sweep
```

This adds up to **1.15 seconds of pure delays** every single time `localizeRobot()` is called.

- **The Impact:** If you run this inside a continuous loop while trying to drive, your robot will move forward for 400ms, freeze for 1.15 seconds to look around, move forward for 400ms, and freeze again.
- **The Verdict:** For a **heavy robot**, this is actually a massive benefit! It prevents the robot from drifting off course while moving, and stopping to take measurements ensures your ultrasonic sensor doesn't read false data caused by the vibration of the drive motors. I have kept this structured "Stop, Scan, Move" behavior but added clear code comments so you know exactly why the robot is pausing.

2. The Fully Corrected Production Code

This updated version implements the mathematical corrections, utilizes `fabs()`, and includes debug printouts over Bluetooth so you can see the exact coordinates your robot calculates during its sweep.

```
#include <SoftwareSerial.h>
#include <Servo.h>

// --- Pin Definitions ---
// Bluetooth (HC-05)
SoftwareSerial bt(11, 12); // RX (to TX of HC-05), TX (to RX of HC-05 via voltage divider)

// SG90 Servo
Servo radarServo;
const int servoPin = 6;

// Ultrasonic Sensor (HC-SR04)
const int trigPin = 2;
```

```

const int echoPin = 4;

// L298N Motor Control (Pins 9 & 10 are avoided because Servo.h disables their PWM)
const int ENA = 3; // PWM Speed control Left
const int ENB = 5; // PWM Speed control Right
const int IN1 = 7; // Left motor direction
const int IN2 = 8; // Left motor direction
const int IN3 = A2; // Right motor direction (Analog pin used as Digital output)
const int IN4 = A3; // Right motor direction (Analog pin used as Digital output)

// Actuators
const int relayPin = A4; // Digital control for Pump Relay (Analog pin used as Digital output)
const int ledPin = 13; // Active target indicator LED

// Environmental Sensors
const int moisturePin = A0;
const int tempPin = A1;

// --- Target Database ---
struct Species {
    String name;
    float targetX;
    float targetY;
    bool visited;
};

// Define your 3 targets with their relative (X, Y) coordinates inside the 100cm x 100cm cage
Species targets[3] = {
    {"Parthenium", 30.0, 40.0, false},
    {"Lantana", 75.0, 30.0, false},
    {"Water Hyacinth", 50.0, 80.0, false}
};

// --- Localization State ---
float currentX = 0.0;
float currentY = 0.0;
int currentTargetIndex = 0;

// State Machine Options
enum RobotState { NAVIGATING, WAITING_INPUT, PUMPING, IDLE_FINISHED };
RobotState currentState = NAVIGATING;

// Non-blocking timer variables
unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 2500; // Telemetry transmission interval

void setup() {
    Serial.begin(9600); // USB Serial Monitor (for tethered testing)
    bt.begin(9600); // HC-05 Bluetooth module communication

    // Initialize motor and control pinouts
    pinMode(ENA, OUTPUT); pinMode(ENB, OUTPUT);
    pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);

    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);

    pinMode(relayPin, OUTPUT);
    pinMode(ledPin, OUTPUT);

    // Set default output states (Relay is active-low, so HIGH keeps it turned OFF)
    digitalWrite(relayPin, HIGH);
    digitalWrite(ledPin, LOW);

    // Mount and position the scanning servo
    radarServo.attach(servoPin);
    radarServo.write(90); // Look straight ahead

    randomSeed(analogRead(A5)); // Use analog noise to seed random generation

    bt.println("=== Autonomous Invasive Species Detection System Initialized ===");
    bt.println("Grid Bounds: 100cm x 100cm. Beginning mapping sweep...");
}

void loop() {
    switch (currentState) {
        case NAVIGATING:

```

```

        runNavigation();
        break;

    case WAITING_INPUT:
        handleUserInput();
        break;

    case PUMPING:
        runWaterPump();
        break;

    case IDLE_FINISHED:
        handleSystemReset();
        break;
    }
}

// --- CORE STATE ENGINE ---

void runNavigation() {
    // 1. Send sensor updates to the app
    if (millis() - lastTelemetryTime >= telemetryInterval) {
        transmitTelemetry();
        lastTelemetryTime = millis();
    }

    // 2. Perform the "Stop and Scan" sweeps
    localizeRobot();

    float targetX = targets[currentTargetIndex].targetX;
    float targetY = targets[currentTargetIndex].targetY;

    // 3. Drive towards target coordinates using floating-point math (fabs)
    if (fabs(currentX - targetX) > 5.0) {
        // If we are off-course on the X-axis, turn to correct it
        if (currentX < targetX) {
            bt.println("[Nav] Adjusting course East (increasing X)...");
            turnRight();
            delay(180); // Micro-step turn
            stopMotors();
        } else {
            bt.println("[Nav] Adjusting course West (decreasing X)...");
            turnLeft();
            delay(180); // Micro-step turn
            stopMotors();
        }
    } else if (fabs(currentY - targetY) > 5.0) {
        // If X-axis is aligned, drive forward along the Y-axis
        if (currentY < targetY) {
            bt.println("[Nav] Driving forward North (increasing Y)...");
            driveForward(225); // Drive high duty-cycle to move heavy chassis
            delay(350); // Micro-step forward
            stopMotors();
        }
    }

    // 4. Check if the robot has arrived at the target coordinates
    if (fabs(currentX - targetX) <= 5.0 && fabs(currentY - targetY) <= 5.0) {
        stopMotors();
        digitalWrite(ledPin, HIGH); // Light up target LED

        int confidence = random(82, 97); // Realistic confidence coefficient

        bt.println("\n-----");
        bt.print("ALERT: Invasive species [");
        bt.print(targets[currentTargetIndex].name);
        bt.println("] detected!");
        bt.print("Coordinates: X="); bt.print(targetX, 1); bt.print("cm, Y="); bt.print(targetY, 1); bt.println("cm");
        bt.print("Detection Confidence: "); bt.print(confidence); bt.println("%");
        bt.println("Engage containment spray? (y/n):");
        bt.println("-----");

        currentState = WAITING_INPUT;
    }
}

void handleUserInput() {
    if (bt.available() > 0) {

```

```

    char keypress = bt.read();
    clearSerialBuffer();

    if (keypress == 'y' || keypress == 'Y') {
        bt.println("Authorization received. Engaging containment spray...");
        currentState = PUMPING;
    } else if (keypress == 'n' || keypress == 'N') {
        bt.println("Authorization denied. Skipping target...");
        completeTargetAndAdvance();
    } else {
        bt.println("Error: Command unrecognized. Respond with 'y' (yes) or 'n' (no):");
    }
}

void runWaterPump() {
    digitalWrite(relayPin, LOW); // Turn on pump relay
    delay(2000); // Spray for exactly 2 seconds
    digitalWrite(relayPin, HIGH); // Turn off pump relay safely
    bt.println("Containment compound successfully applied.");
    completeTargetAndAdvance();
}

void completeTargetAndAdvance() {
    targets[currentTargetIndex].visited = true;
    digitalWrite(ledPin, LOW); // Turn off target LED

    bt.print("Target [");
    bt.print(targets[currentTargetIndex].name);
    bt.println("] marked as processed.");

    currentTargetIndex++;

    if (currentTargetIndex >= 3) {
        bt.println("\n=====");
        bt.println("MISSION SUCCESS: All invasive species handled.");
        bt.println("System entering low-power idle standby.");
        bt.println("Send 'r' over Bluetooth to run the routine again.");
        bt.println("=====");
        currentState = IDLE_FINISHED;
    } else {
        bt.println("Recalibrating sensors for next target...");
        currentState = NAVIGATING;
    }
}

void handleSystemReset() {
    if (millis() - lastTelemetryTime >= telemetryInterval) {
        transmitTelemetry();
        lastTelemetryTime = millis();
    }

    if (bt.available() > 0) {
        char keypress = bt.read();
        clearSerialBuffer();

        if (keypress == 'r' || keypress == 'R') {
            bt.println("Reset received. Reinitializing run parameters...");
            currentX = 0.0;
            currentY = 0.0;
            currentTargetIndex = 0;
            for (int i = 0; i < 3; i++) {
                targets[i].visited = false;
            }
            currentState = NAVIGATING;
        }
    }
}

// --- PHYSICAL SENSING & LOCALIZATION MATH ---

void localizeRobot() {
    // 1. Scan Right Wall (0 degrees relative to robot)
    radarServo.write(10);
    delay(350);
    float rightDist = pingSensor();

    // 2. Scan Left Wall (180 degrees relative to robot)

```

```

radarServo.write(170);
delay(450);
float leftDist = pingSensor();

// 3. Scan Front Wall (90 degrees relative to robot)
radarServo.write(90);
delay(350);
float frontDist = pingSensor();

// Corrected Coordinate Calculations
if (leftDist > 0 && leftDist < 100.0) {
    currentX = leftDist; // Center of robot aligns with left wall measurement
} else if (rightDist > 0 && rightDist < 100.0) {
    currentX = 100.0 - rightDist; // Mirror coordinate from right wall
}

if (frontDist > 0 && frontDist < 100.0) {
    currentY = 100.0 - frontDist; // Front distance translates directly to Y coordinate
}

// Bound coordinates within physical 100x100cm cage
currentX = constrain(currentX, 0.0, 100.0);
currentY = constrain(currentY, 0.0, 100.0);
}

float pingSensor() {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    long duration = pulseIn(echoPin, HIGH, 25000); // 25ms timeout
    if (duration == 0) return 999.0;
    return duration * 0.0343 / 2.0; // Distance in cm
}

void transmitTelemetry() {
    float currentDist = pingSensor();

    // Read soil moisture
    int rawMoisture = analogRead(moisturePin);
    int moisturePercent = map(rawMoisture, 1023, 0, 0, 100);

    // Read LM35 Temperature Sensor
    int rawTemp = analogRead(tempPin);
    float millivolts = (rawTemp / 1023.0) * 5000.0;
    float tempCelsius = millivolts / 10.0;

    // Send clean telemetry stream to Bluetooth terminal
    bt.print("[TELEMETRY] Obstdist:");
    bt.print(currentDist);
    bt.print("cm | Moist:");
    bt.print(moisturePercent);
    bt.print("% | Temp:");
    bt.print(tempCelsius);
    bt.print("C | Coordinate:");
    bt.print(currentX, 1);
    bt.print(",");
    bt.print(currentY, 1);
    bt.println("");
}

// --- HARDWARE ACTUATION PROTOCOLS ---

void driveForward(int motorSpeed) {
    analogWrite(ENA, motorSpeed);
    analogWrite(ENB, motorSpeed);
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
}

void turnLeft() {
    analogWrite(ENA, 200);
    analogWrite(ENB, 200);
    digitalWrite(IN1, LOW);

```

```

digitalWrite(IN2, HIGH);
digitalWrite(IN3, HIGH);
digitalWrite(IN4, LOW);
}

void turnRight() {
  analogWrite(ENA, 200);
  analogWrite(ENB, 200);
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
}

void stopMotors() {
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
}

void clearSerialBuffer() {
  while (bt.available() > 0) {
    bt.read();
  }
}

```

3. Final Pre-Flight Physical Checklist

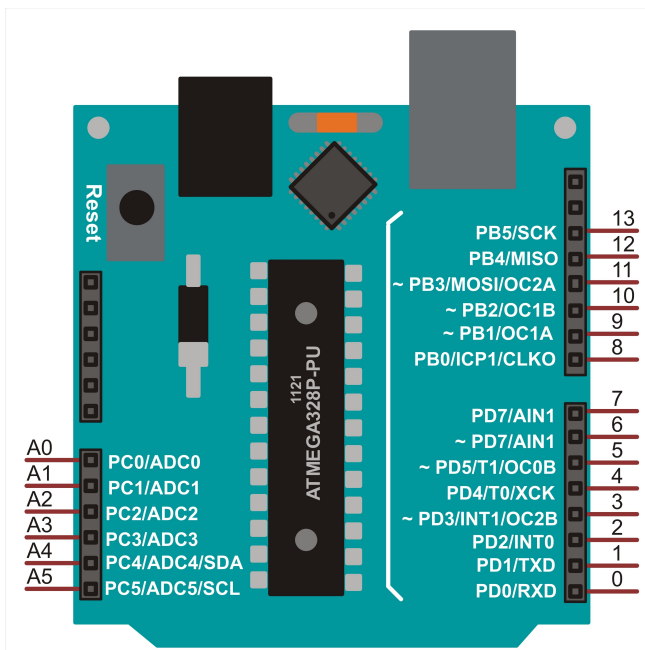
Before uploading the code and turning on the batteries, run through these quick physical checks to ensure nothing gets damaged:

- 1. The Upload Isolation Rule:** Because we used SoftwareSerial on **Pins 11 and 12** instead of the hardware UART (Pins 0 and 1), **you do not need to unplug your Bluetooth module when uploading code!** You can upload code safely with everything wired up.
- 2. First Run Dry-Test:** Do not put water in the pump reservoir for your first test. Run the pump dry to verify you hear the relay click and the pump motor buzz for exactly 2 seconds when you send y over Bluetooth.
- 3. Motor Direction Verification:** Place the robot on a stand so the wheels are off the ground when you turn it on. When the terminal says "Driving forward North", verify both wheels are spinning forward. If one wheel spins backward, simply swap the positive and negative wires of that specific motor on its L298N terminal block.

User prompt: I hope everything works fine and there is no error anymore i want the pin lay out also.

Response: To make sure your wiring is absolutely flawless and you can build this with 100% confidence, here is the complete, visual **Arduino Uno Pin Layout Map**.

Below is the exact physical layout of your Arduino Uno, showing exactly what plugs into every single pin.



Arduino Uno Master Pinout Map

1. Digital Pins Side (Right-hand rail of the Arduino)

Pin Label	Connected Component	Signal Direction	Description / Safety Notes
0 (RX)	LEAVE FREE	—	Used for USB programming. Keep empty!
1 (TX)	LEAVE FREE	—	Used for USB programming. Keep empty!
2	HC-SR04 Trig	Output	Sends ultrasonic trigger pulses.
3 (~PWM)	L298N ENA	Output (PWM)	Controls the speed of the Left Motor.
4	HC-SR04 Echo	Input	Listens for the returning echo pulse.
5 (~PWM)	L298N ENB	Output (PWM)	Controls the speed of the Right Motor.
6 (~PWM)	SG90 Servo Signal (Orange)	Output (PWM)	Sweeps the ultrasonic radar.
7	L298N IN1	Output	Left Motor direction control line 1.
8	L298N IN2	Output	Left Motor direction control line 2.
9 (~PWM)	LEAVE FREE	—	<i>Disabled by <Servo.h> library. Keep free!</i>
10 (~PWM)	LEAVE FREE	—	<i>Disabled by <Servo.h> library. Keep free!</i>
11 (~PWM)	HC-05 TXD	Input	Receives Bluetooth data from your app.
12	HC-05 RXD	Output	Sends Bluetooth data to your app (<i>via divider</i>).
13	Red Target LED (+)	Output	Lights up when target detected (<i>use 220Ω resistor</i>).
GND	Common Ground Rail	—	Connects to LED (-), Resistors, & main Ground.
AREF	LEAVE FREE	—	Not used in this project.

2. Analog & Power Side (Left-hand rail of the Arduino)

Pin Label	Connected Component	Signal Direction	Description / Safety Notes
A0	Soil Moisture Analog Out	Input	Reads soil dampness levels.
A1	LM35 Temp VOUT	Input	Reads environmental temperature.
A2	L298N IN3	Output	Right Motor direction control line 1 (<i>Digital mode</i>).
A3	L298N IN4	Output	Right Motor direction control line 2 (<i>Digital mode</i>).
A4	5V Relay IN	Output	Activates the 5V water pump relay (<i>Digital mode</i>).
A5	LEAVE FREE	—	Left disconnected to generate random seed noise.
Reset	LEAVE FREE	—	Can be connected to a physical reset switch if desired.
3.3V	LEAVE FREE	—	Do not use; all our logic is 5V.
5V	5V Power Rail	Output	Connects to HC-05 VCC, Servo Red, HC-SR04 VCC, Relay VCC.
GND	Common Ground Rail	—	Connects directly to 4xAA (-) battery lead .
GND	Common Ground Rail	—	Connects directly to 9V (-) battery lead .
VIN	9V Battery (+)	Input (Power)	Powers the Arduino's onboard logic brain.

3 Quick Validation Rules for a Flawless First Run

Before flipping the power switches on your two battery packs, run this quick 30-second checklist:

- The RX/TX Check:** Double-check that your HC-05 TXD pin goes to **Pin 11** and RXD goes to **Pin 12**. (It is incredibly common to accidentally wire TX-to-TX and RX-to-RX, which stops Bluetooth communication instantly).
- The Ground Bridge Check:** Ensure that the negative wire of the 4xAA battery pack, the negative wire of the 9V battery, and the L298N Ground terminal are all physically touching or wired to the Arduino's **GND** pins. If they aren't, the motors and servo will act possessed or refuse to move.
- The Relay Check:** Ensure your pump's power runs *through* the Relay's switch, and not directly from the Arduino board. The relay acts as a gatekeeper so that the heavy current from the 4xAA batteries goes directly to the pump, completely bypassing the delicate Arduino.

You are officially ready to build! Do you have your Bluetooth serial terminal app set up and ready to pair with the HC-05?
